

 SAYMAN BLOCKCHAIN — PHASE 21: EVM WALLET INTEGRATION & PARALLEL ARCHIVE SYNCHRONIZATION

JavaScript-native Smart Contracts · Proof-of-Stake · Multi-Layer Chains · Custom Tokens · NFTs · DEX · Memecoins · Staking Pools

[!Phase](https://img.shields.io/badge/Phase-21-brightgreen)](https://github.com/saymanlal/SAYMAN)

[!Network](https://img.shields.io/badge/Network-Public%20Testnet-blue)](https://sayman.onrender.com)

[!License](https://img.shields.io/badge/License-MIT-yellow)](LICENSE)

What's New in Phase 21

Phase 21 introduces **native EVM JSON-RPC integration** for global wallet support (MetaMask, Trust Wallet, etc.), **parallelized block archive synchronization** (concurrency limit = 15) to catch up to 25K+ blocks in seconds on startup or crash recovery, and critical **LevelDB memory optimization** to eliminate continuous OOM crashes on Render.

| Feature | Status |

|---|---|

| **EVM JSON-RPC Support** |  Mounts standard Ethereum JSON-RPC endpoints at `/`, `/api`, and `/rpc` (supporting `eth_chainId`, `net_version`, `eth_blockNumber`, `eth_getBlockByNumber`, `eth_getBalance`, `eth_getTransactionCount`, `eth_sendRawTransaction`, etc.) to work out-of-the-box with MetaMask and other wallets globally |

| **Parallel Archive Sync** |  Implements a concurrency-safe promise pool (limit = 15) to fetch block chunks from the GitHub archive in parallel, reducing sync times from hours to seconds |

| **LevelDB Storage Fix** |  Removes full-chain serialization to the LevelDB `'chain'` key on every block, eliminating the $O(N^2)$ memory bottleneck and preventing OOM crashes on Render |

| **Direct State Snapshots** |  Downloads and writes state snapshots directly to the local snapshot directory (`snapshot-${targetHeight}.json`) to accelerate node initialization |

MetaMask & Global Wallet Integration

SAYMAN supports standard Ethereum JSON-RPC 2.0. You can connect MetaMask, Trust Wallet, Coinbase Wallet, or any other Web3 wallet globally to SAYMAN using the following custom network parameters:

Network Connection Settings

| Parameter | Public Testnet | Local Development |

| :--- | :--- | :--- |

| **Network Name** | Sayman Public Testnet | Sayman Local Testnet |

| **New RPC URL** | `https://sayman.onrender.com` | `http://localhost:10000` |

| **Chain ID** | 82922 (or hex `0x143ea`) | 82923 (or hex `0x143eb`) |

| **Currency Symbol** | SAYN | SAYN |

| **Block Explorer** | `https://sayman.onrender.com` | `http://localhost:10000` |

Supported JSON-RPC API Methods

The node exposes an EVM-compatible RPC layer at `/`, `/api`, and `/rpc` supporting:

- `eth_chainId` / `net_version` - returns the network numeric identifier (82922 for public testnet)
- `eth_blockNumber` - returns the latest block index
- `eth_getBlockByNumber` / `eth_getBlockByHash` - returns block data in EVM format
- `eth_getBalance` - retrieves address balance mapped to Wei (multiplied by 10^{10} \$)
- `eth_getTransactionCount` - retrieves account nonces for sequential anti-replay transactions
- `eth_sendRawTransaction` - accepts EIP-155, EIP-2930, and EIP-1559 signed Ethereum transactions, recovers the sender public key using ECDSA (secp256k1), and broadcasts it to SAYMAN mempool
- `eth_getTransactionByHash` / `eth_getTransactionReceipt` - returns transaction details and receipts
- `eth_estimateGas` - returns estimated gas (21000 gas base limit for standard transactions)

Architecture

```
sayman/
```

```
└─ core/
```

```

|   └─ blockchain.js      # PoS engine (TPS, gasLimits/gasCosts in get
|
|   └─ chain-factory.js   #  ChainFactory: L2/Sidechain/Permissioned
|
|   └─ rollup.js          #  L2 state root commitment to L1
|
|   └─ state.js           # StateEngine: balances, stakes, reputation,
|
|   └─ contracts.js       # Sandboxed JS VM
|
|   └─ gas.js             # Gas calculator
|
|   └─ ...
|
└─ contracts/
|
|   └─ token.js           # ERC-20 style token
|
|   └─ nft.js             # ERC-721 style NFT
|
|   └─ token-factory.js   # Token factory
|
|   └─ nft-factory.js     # NFT collection factory
|
|   └─ memecoin-factory.js #  Memecoin launcher (burn/tax/anti-whale)

```

└─ dex.js	#  AMM DEX (Uniswap V2 style)
└─ staking-pool.js	#  Delegated staking pool
└─ layer2Bridge.js	#  L1 bridge for L2 deposits/withdrawals
└─ p2p/server.js	# P2P sync + peer reputation points
└─ api/routes.js	# REST API (12+ endpoints added in Phase 14)
└─ frontend/	# Explorer UI (Layers page, TPS card, denomi
└─ config/	# Network configs (testnet, public-testnet, i
└─ server.js	# Server (validator + sequencer modes)



Tokenomics & Denomination

Property	Value
----------	-------

---	---
-----	-----

Ticker	
--------	---

Base Unit	sprinkle (1 SAYN = 100,000,000 sprinkles)
-----------	---

Decimals	8
----------	---

Block Time	5 seconds
------------	-----------

| Block Reward (Testnet) | 0.5 SAYN = 50,000,000 sprinkles |

| Block Reward (Mainnet) | 0.2 SAYN = 20,000,000 sprinkles |

| Min Stake (Testnet) | 10 SAYN = 1,000,000,000 sprinkles |

| Min Stake (Mainnet) | 500 SAYN = 50,000,000,000 sprinkles |

> **API Clarity:** All on-chain amounts are integers in base units (sprinkles).

> Call `GET /api/denomination` for the conversion table, or check `/api/tps` which always returns `decimals` and `ticker`.

Multi-Layer Chains

Anyone can spin up their own chain using the `ChainFactory`:

```
import { ChainFactory } from './core/chain-factory.js';

import Blockchain from './core/blockchain.js';

// Layer 2 Rollup (anchored to SAYMAN L1)

const config = ChainFactory.createL2Config({

  name: 'MyL2Chain',

  chainId: 'my-l2-1',

  l1RpcUrl: 'https://sayman.onrender.com',

  l1Bridge: '<BRIDGE_CONTRACT_ADDRESS>',

  genesis: { '<YOUR_ADDRESS>': 1_000_000_000 }
```

```
});

// Independent Sidechain

const sideConfig = ChainFactory.createSidechainConfig({

  name: 'GameChain',

  chainId: 'gamechain-1',

  blockReward: 10_000_000,

});

// Permissioned (private consortium)

const permConfig = ChainFactory.createPermissionedConfig({

  name: 'EnterpriseChain',

  chainId: 'ent-1',

  validators: ['<ADDR1>', '<ADDR2>']

});
```

Running Locally

Quick Start

```
git clone https://github.com/saymanlal/SAYMAN
```

```
cd SAYMAN
```

```
npm install
```

Start validator node

```
node server.js --network testnet --mode validator
```

Start L2 sequencer (commits to L1 ev

```
node server.js --network testnet --mode sequencer
```

Sync-only node

```
node server.js --network testnet --mode observer
```

Environment Variables

```
PORT=10000
```

```
NETWORK=public-testnet
```

```
MODE=validator
```

```
BOOTSTRAP_PEERS=wss://peer1.example.com/p2p,wss://peer2.example.com/p2p
```

```
DB_PATH=./data/node-10000
```

For L2 sequencer mode:

```
L1_RPC_URL=https://sayman.onrender.com
```

```
L1_BRIDGE_CONTRACT=<contract_address>
```

```
L1_SEQUENCER_PRIVATE_KEY=<hex_private_key>
```

Open `http://localhost:10000` after starting the node. The built-in explorer includes:

-  Dashboard with TPS, denomination guide, APR, mempool
-  Block Explorer with search + pagination
-  Validators with stake (SAYN + base units), reputation, missed blocks
-  Smart Contracts registry
-  Layers page — how to create your own chain
-  Network — peers, node info, uptime

REST API

Full reference is in `AI.md`. Key endpoints:

```
# Network info (layer, decimals, denomination)

curl https://sayman.onrender.com/api/network
```

SAYN/base-unit conversion guide

```
curl https://sayman.onrender.com/api/denomination
```

Live TPS

```
curl https://sayman.onrender.com/api/tps
```

Custom tokens

```
curl https://sayman.onrender.com/api/tokens
```

NFT collections

```
curl https://sayman.onrender.com/api/nfts
```

Staking pools

```
curl https://sayman.onrender.com/api/staking-pools
```

Account (balance in base units + rep

```
curl https://sayman.onrender.com/api/address/<address>
```

```
curl -X POST https://sayman.onrender.com/api/broadcast \

-H "Content-Type: application/json" \

-d '{"type":"TRANSFER","data":{"..."},"signature":"...","publicKey":"..."}'
```

— — —



```
// Deploy token factory, then create your own token:

client.callContract({

contractAddress: TOKEN_FACTORY_ADDR,


method: 'createToken',


args: { name: 'DOGE ON SAYMAN', symbol: 'SDOGE', totalSupply: 1_000_000_000_000_000_000n },


});
```

```
// Launch a memecoin with tax + anti-whale:

client.callContract({

  contractAddress: MEMECOIN_FACTORY_ADDR,

  method: 'launch',

  args: {

    name: 'PepeSAYN', symbol: 'PSAYN', totalSupply: 420_000_000_000,

    transferTaxPercent: 2, maxWalletPercent: 1, burnOnTransfer: true

  }

});

// Add liquidity to DEX:

client.callContract({

  contractAddress: DEX_ADDR,

  method: 'addLiquidity',
```

```
args: { tokenA: 'ADDR_A', tokenB: 'ADDR_B', amountA: 1000, amountB: 2000  
  
});
```



PUKY Mobile Wallet

Available as an Android APK at `/apk/puky.apk` . Features:

- Multi-RPC failover (Railway + Render + local)
- Real-time GitHub APK update checker
- Send/receive SAYN, stake, unstake



Contributing

See `ABOUT.md` for full project vision. PRs welcome on `main` or `phase20` branch.

Phase 20 Updates

- **Integrated Transaction Explorer:** Added dedicated transactions page, statistics bar, and rewards filter (toggles REWARD/REWARD_FEE txs).
- **Archive System Concurrency and Stability:** Introduced single-writer lock, exponential retry backoff, and sync-state pause protections.
- **Repository Cleanups:** Purged legacy CLI, SDK, and Faucet code to complete separation of concerns.



License

MIT — see `LICENSE` file.

Links

- **Explorer:** <https://sayman.onrender.com>
- **Wallet:** <https://sayman-wallet-manager.vercel.app>
- **Faucet:** <https://sayman-faucet-site.vercel.app>
- **GitHub:** <https://github.com/saymanlal/SAYMAN>

<div style="page-break-after: always;"></div>

 SAYMAN Blockchain — Network Info & Wallet Connection Guide

Phase 21 | Public Testnet | Chain ID: 82922 **RPC Endpoint:**

<https://sayman.onrender.com> **Block Explorer:** <https://sayman.onrender.com> **GitHub:**

<https://github.com/saymanlal/SAYMAN>

Connect Your Wallet (MetaMask & All Global Wallets)

SAYMAN supports the standard **Ethereum JSON-RPC 2.0** protocol. Any wallet that accepts custom networks (MetaMask, Trust Wallet, Coinbase Wallet, Rainbow, Rabby, imToken, Exodus, Brave Wallet, etc.) can connect using the parameters below.

Public Testnet (Live Network)

Parameter	Value
---	---
Network Name	Sayman Public Testnet
New RPC URL	https://sayman.onrender.com
Chain ID	82922
Currency Symbol	SAYN
Block Explorer URL	https://sayman.onrender.com
Network Hex ID	0x143ea

Local Development Node

Parameter	Value
-----------	-------

---|---

| **Network Name** | Sayman Local Testnet |

| **New RPC URL** | http://localhost:10000 |

| **Chain ID** | 82923 |

| **Currency Symbol** | SAYN |

| **Block Explorer URL** | http://localhost:10000 |

Step-by-Step: Add SAYMAN to MetaMask

Desktop (MetaMask Browser Extension)

1. Open MetaMask → click the network dropdown at the top
2. Click **"Add network"**
3. Click **"Add a network manually"** at the bottom
4. Fill in the fields:
 - **Network name:** Sayman Public Testnet
 - **New RPC URL:** https://sayman.onrender.com
 - **Chain ID:** 82922
 - **Currency symbol:** SAYN
 - **Block explorer URL:** https://sayman.onrender.com
5. Click **Save** → then **"Switch to Sayman Public Testnet"**

Mobile (MetaMask App)

1. Open MetaMask → tap the hamburger menu
2. Tap **Settings** → **Networks** → **Add Network**
3. Enter the same values as above
4. Tap **Add** to confirm



Add to Other Wallets

Trust Wallet

1. Go to Settings → Preferences → Custom Network
2. Enter: RPC URL `https://sayman.onrender.com` , Chain ID `82922` , Symbol `SAYN`

Coinbase Wallet

1. Settings → Active Networks → Add Custom Network
2. Enter the same network details as above

Rabby / Rainbow / Brave Wallet

- All support EIP-3085 (`wallet_addEthereumChain`) — use the same values

WalletConnect

- SAYMAN is fully WalletConnect compatible; use Chain ID `82922` to connect



Supported JSON-RPC Methods

| Method | Description |

|---|---|

| `eth_chainId` | Returns `0x143ea` (82922 for public testnet) |

| `net_version` | Returns `"82922"` |

| `eth_blockNumber` | Latest block height in hex |

| `eth_getBlockByNumber` | Block data (EVM-format) by number |

| `eth_getBlockByHash` | Block data (EVM-format) by hash |

| `eth_getBalance` | Address balance in Wei-equivalent (base units × 10¹⁰) |

| `eth_getTransactionCount` | Account nonce (for transaction ordering) |

| `eth_sendRawTransaction` | Submit signed EIP-155/2930/1559 transaction |

| `eth_getTransactionByHash` | Transaction details by hash |

| `eth_getTransactionReceipt` | Transaction receipt (status, gas used) |

| `eth_estimateGas` | Gas estimate (returns `0x5208` = 21000) |

| `eth_gasPrice` | Current gas price in Wei-equivalent |

| `eth_accounts` | Returns `[]` (non-custodial) |

| `web3_clientVersion` | `SAYMAN/v21.0.0/javascript` |



Example: curl JSON-RPC Calls

```
# Get chain ID

curl -X POST https://sayman.onrender.com \

-H "Content-Type: application/json" \

-d '{"jsonrpc":"2.0","method":"eth_chainId","params":[],"id":1}'
```

Get latest block number

```
curl -X POST https://sayman.onrender.com \

-H "Content-Type: application/json" \

-d '{"jsonrpc":"2.0","method":"eth_blockNumber","params":[],"id":1}'
```

Get balance (replace ADDRESS with yo

```
curl -X POST https://sayman.onrender.com \

-H "Content-Type: application/json" \

-d '{"jsonrpc":"2.0","method":"eth_getBalance","params":["0xADDRESS","la
```

Network Overview (Phase 21)

Parameter	Value

Network Name	Sayman Public Testnet
Chain ID (string)	sayman-public-testnet-1
Chain ID (numeric)	82922
Layer	L1 (Base chain)
Block Time	5 seconds
Block Reward	0.5 SAYN (50,000,000 base units)
Min Stake	10 SAYN (1,000,000,000 base units)
Max Supply	Unlimited (testnet)
Ticker	SAYN
Base Unit	sprinkle (1 SAYN = 100,000,000 sprinkles)

| **Decimals** | 8 |

| **Gas Price** | 1 base unit per gas unit |

| **Transfer Gas** | 21,000 gas |

| **Faucet** | ☒ Enabled |

| **Faucet Amount** | 1,000 SAYN per drip |

| **Faucet Cooldown** | 60 seconds |



Gas Model

Gas Limits

| Parameter | Value |

---|---

| **Min Gas Price** | 1 wei |

| **Max Gas Per Transaction** | 10,000,000 |

| **Max Gas Per Block** | 100,000,000 |

| **Max Execution Time** | 5,000ms |

| **Max State Size** | 500KB |

| **Max Instructions** | 100,000 |

Gas Costs

| Transaction Type | Base Gas Cost |

---|---

| **TRANSFER** | 21,000 |

| **STAKE** | 50,000 |

| **UNSTAKE** | 50,000 |

| **CONTRACT_DEPLOY** | 200,000 + code size |

| **CONTRACT_CALL** | 50,000 + execution |

| **CONTRACT_UPGRADE** | 300,000 |

| **STORAGE_READ** | 500 |

| **STORAGE_WRITE** | 2,000 |

Security Features

Anti-Spam Protections

-  **Nonce System:** Sequential nonce per address prevents replay attacks
-  **Gas Fees:** Economic cost to submit transactions (nearly free on testnet)
-  **Mempool Limit:** Maximum 1,000 pending transactions
-  **Rate Limiting:** Max 10 transactions per minute per address
-  **EVM Signature Recovery:** ECDSA secp256k1 sender recovery for EIP-155 raw tx

Consensus Security

-  **Proof of Stake:** Validator selection by staked SAYN balance
-  **Minimum Stake:** 10 SAYN required per validator
-  **Slashing:** Validators penalized for missed blocks
-  **Chain ID Validation:** Prevents cross-network replay attacks
-  **State Root Validation:** Deterministic state root per block

Economics

Token Supply

- **Max Supply:** Unlimited (testnet)
- **Block Reward:** 0.5 SAYN per block
- **Blocks Per Day:** 17,280 (at 5s blocktime)
- **Daily Emission:** 8,640 SAYN/day

Staking

- **Minimum Stake:** 10 SAYN (1,000,000,000 base units)
- **Unstake Delay:** 50 blocks
- **Slash Percentage:** 10% for missed blocks

Bootstrap Peers (P2P Network)

Node	URL

Primary (Railway)	<code>wss://sayman.up.railway.app/p2p</code>
Render Node 1	<code>wss://sayman.onrender.com/p2p</code>
Render Node 2	<code>wss://sayman-1.onrender.com/p2p</code>

|---|---|

REST API Endpoints

Base URL: `https://sayman.onrender.com/api`

Method	Endpoint	Description
--- --- ---		
GET	<code>/network</code>	Network configuration
GET	<code>/stats</code>	Network statistics
GET	<code>/blocks</code>	List blocks (paginated)
GET	<code>/block/:index</code>	Get block by index
GET	<code>/block/hash/:hash</code>	Get block by hash
GET	<code>/transactions/:id</code>	Get transaction
GET	<code>/address/:address</code>	Get address info + tx history
GET	<code>/validators</code>	List validators
GET	<code>/contracts</code>	List contracts
GET	<code>/mempool</code>	Current mempool
GET	<code>/tps</code>	Live TPS estimate
GET	<code>/denomination</code>	SAYN/sprinkle conversion table

| POST | `/broadcast` | Submit signed transaction |

| POST | `/estimate-gas` | Estimate gas cost |

| GET | `/faucet/:address` | Request test tokens |

| GET | `/search/:query` | Search block/tx/address |



Archive & Recovery

SAYMAN uses a GitHub-backed archive system (`github.com/saymanlal/sayman-archive`) for block persistence:

- Blocks are archived in chunks of 1,000 blocks per file
- State snapshots (`snapshot-<height>.json`) saved every 100 blocks
- On startup, nodes sync from archive using **batched parallel download** (10 concurrent chunks)
- Nodes can recover **billions of blocks** purely from GitHub archive in seconds/minutes
- After crash & restart, archive data loaded in **< 10 seconds** from CDN



Getting Started

1. Run a Node

```
git clone https://github.com/saymanlal/SAYMAN  
  
cd SAYMAN  
  
  
npm install  
  
  
node server.js --network public-testnet --mode validator
```

2. Get Test Tokens (Faucet)

```
curl -X POST https://sayman.onrender.com/api/faucet \

-H "Content-Type: application/json" \

-d '{"address": "YOUR_ADDRESS"}'
```

3. Become a Validator

```
# Send a STAKE transaction with minimum 10 SAYN
```

Use the PUKY mobile wallet or API di

4. Connect MetaMask

- Open MetaMask → Add Network → Manual
- RPC: `https://sayman.onrender.com` , Chain: `82922` , Symbol: `SAYN`



Current Status (Phase 21)

- **Network Health:** Operational
- **Phase:** 21 — EVM RPC Wallet & Parallel Sync
- **P2P Archive Sync:** Batched parallel (10 concurrent chunks)
- **Memory Model:** ChainProxy — only last 100 blocks in RAM, rest in LevelDB
- **OOM Crashes:** Eliminated — no full-chain write on every block
- **MetaMask Support:** Full EVM JSON-RPC 2.0

Last updated: Phase 21 — July 2026

*Generated by: `npm run docs` → `NETWORK_INFO.md` *

Sayman Phase 21 - Complete Installation Guide

Prerequisites

- **Node.js:** v20.0.0 or higher
- **npm:** v9.0.0 or higher
- **Git:** For cloning repository
- **Terminal:** Bash/Zsh (Linux/Mac) or WSL (Windows)

Check versions:

```
node --version    # Should be >= 20.0.0

npm --version     # Should be >= 9.0.0
```

Quick Start (5 Minutes)

```
# 1. Clone or navigate to project

cd sayman-blockchain
```

2. Install dependencies

```
npm install
```

3. Run testnet

```
npm run testnet
```

4. Open browser

http://localhost:3000

5. Install CLI (optional)

```
npm run install-cli
```

Detailed Installation

Step 1: Install Node.js

Linux (Ubuntu/Debian):

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -  
  
sudo apt-get install -y nodejs
```

Mac:

```
brew install node@20
```

Windows:

Download from: <https://nodejs.org/>

Step 2: Clone Project

```
git clone https://github.com/yourusername/sayman-blockchain.git  
  
cd sayman-blockchain
```

Or if you have the files:

```
cd sayman-blockchain
```

Step 3: Install Dependencies

```
npm install
```

This installs:

- `express` - Web server
- `elliptic` - Cryptography
- `level` - Database
- `uuid` - IDs
- `ws` - WebSocket
- `node-fetch` - HTTP client

Step 4: Install CLI Tool (Optional)

```
npm run install-cli
```

Or manually:

```
cd cli  
  
npm install  
  
npm link  
  
cd ..
```

Verify:

```
sayman --version
```

Should show: 16.0.0

Running the Network

Testnet (Default)

```
npm run testnet
```

Output:

```
┌────────────────────────────────────────────────────────────────────────────────┐  
│ SAYMAN BLOCKCHAIN - PHASE 21 │
```

|| EVM Wallet & Parallel Archive Sync ||

NETWORK: TESTNET

 Network Name: Sayman Testnet

 Chain ID: sayman-testnet-1

 Gas: ENABLED

 Faucet: ENABLED

✓ API server running on http://localhost:3000

Mainnet

```
npm run mainnet
```

Differences:

- No faucet
- Higher minimum stake
- Longer block time
- Production settings

Multi-Node Network

Terminal 1 (Bootstrap):

```
npm run node1
```

Terminal 2 (Peer 1):

```
npm run node2
```

Terminal 3 (Peer 2):

```
npm run node3
```

Access:

- Node 1: <http://localhost:3000>
- Node 2: <http://localhost:3001>
- Node 3: <http://localhost:3002>

Using the CLI

First Time Setup

```
# Create wallet  
  
sayman wallet create
```

Output:



Wallet created successfully!

Address: abc123...def456

Private Key: 0x1234567890abcdef...



Never share your private key with

Your wallet is saved at: ~/.sayman/w

Get Test Tokens (Testnet Only)

Via CLI:

```
# Get your address
```

```
sayman wallet info
```

Request from faucet (via browser or

```
curl -X POST http://localhost:3000/api/faucet \
```

```
-H "Content-Type: application/json" \
```

```
-d '{"address": "YOUR_ADDRESS"}'
```

Via UI:

- Open <http://localhost:3000>
- Click "Faucet"
- Paste your address
- Click "Claim 1000 SAYN"

Check Balance

```
sayman balance
```

Output :

— Account Balance —

Address: **abc123...**

Balance: **1000 SAYN**

Staked: **0 SAYN**

Nonce: **0**

Send Tokens

```
sayman send 0xRECIPIENT_ADDRESS 100
```

With custom gas:

```
sayman send 0xRECIPIENT_ADDRESS 100 --gas-limit 50000 --gas-price 2
```

Output:

— Transaction Details —

From: **abc123...**

To: **def456...**

Amount: **100 SAYN**

Gas Limit: **50000**

Gas Price: 2

Max Cost: 100.0001 SAYN



Transaction broadcast successful

TX ID: 550e8400-e29b-41d4-a716-44665

Stake Tokens

sayman stake 500

Output:

— Staking Transaction —

Address: **abc123...**

Amount: **500 SAYN**

Gas Limit: **100000**

Gas Price: **1**



Stake transaction broadcast!

You will become a validator once the

View Network

sayman network

Output:

— Network Information —

Network: Sayman Testnet

Chain ID: sayman-testnet-1

Block Height:150

Validators: 4

Total Stake: 2000 SAYN

Block Time: 5s

Block Reward: 10 SAYN

Min Stake: 100 SAYN

Faucet: ✓ Enabled

List Validators

```
sayman validators
```

Output :

— Active Validators —

Address	Stake	%
abc123...	500 SAYN	25%
def456...	300 SAYN	15%

| **ghi789...** | **1200 SAYN** | **60%**



Using the Web UI

Dashboard

Open <http://localhost:3000>

Features:

- Live network statistics
- Real-time block feed (updates every 5s)
- Validator monitoring
- Gas usage tracking

Create Wallet

1. Click "Wallet" tab
2. Click "Create New Wallet"
3. **SAVE YOUR PRIVATE KEY!**
4. Copy your address

Get Test Tokens

1. Click "Faucet" tab

2. Paste your address
3. Click "Claim 1000 SAYN"
4. Wait ~6 seconds for block

Send Transaction

1. Click "Wallet" tab
2. Scroll to "Send SAYN"
3. Enter recipient address
4. Enter amount
5. Enter your private key
6. Click "Send Transaction"

Note: Gas is calculated automatically.

Stake

1. Click "Wallet" tab
2. Scroll to "Stake SAYN"
3. Enter amount (min: 100 SAYN)
4. Enter your private key
5. Click "Stake"

Deploy Contract

1. Click "Contracts" tab
2. Write contract code:

```
function increment() {  
  
  state.count = (state.count || 0) + 1;  
  
}
```

```
function getCount() {  
  
    return state.count || 0;  
  
}
```

3. Enter your private key
4. Click "Deploy Contract"
5. Wait for confirmation

Call Contract

1. After deployment, copy contract address
2. Scroll to "Call Contract"
3. Paste contract address
4. Enter method: `increment`
5. Arguments: `{}`
6. Enter private key
7. Click "Call Contract"

Generate Documentation

```
npm run docs
```

Generates `NETWORK_INFO.md` in the root directory with:

- Current network stats
- Gas configuration
- Validator list

- API reference
- Economics data

Testing

Run Automated Tests

```
chmod +x test-phase6.sh  
  
./test-phase6.sh
```

Tests:

-  Network configuration
-  Gas model
-  Nonce system
-  API endpoints
-  Documentation generator
-  Mempool limits
-  Validators
-  Contracts

Manual Testing

1. Test Wallet:

```
sayman wallet create  
  
sayman balance
```

2. Test Transactions:

```
# Get test tokens first (via faucet)  
  
sayman send 0xTEST_ADDRESS 10
```

3. Test Staking:

```
sayman stake 100
```

```
sayman validators
```

4. Test Gas Estimation:

```
sayman estimate TRANSFER '{"from":"0x...", "to":"0x...", "amount":100}'
```

5. Test Nonce:

```
# Check nonce
```

```
sayman balance
```

Shows: Nonce: 0

Send transaction

```
sayman send 0x... 10
```

Nonce increments to 1

Check again

```
sayman balance
```

Shows: Nonce: 1

Configuration

Change API Endpoint (CLI)

```
sayman config http://localhost:3001/api
```

Environment Variables

```
# Change port  
PORT=4000 npm run testnet
```

Change P2P port

```
P2P_PORT=7001 npm run testnet
```

Add peers

```
PEERS=ws://peer1:6001,ws://peer2:6002 npm run testnet
```

Use mainnet

```
NODE_ENV=mainnet npm start
```

Custom Network

Edit `config/testnet.js` or `config/mainnet.js`:

```
export default {  
  
  networkName: 'My Custom Network',  
  
  chainId: 'custom-1',  
  
  blockTime: 3000,      // 3 seconds  
  
  blockReward: 20,      // 20 SAYN
```

```
minStake: 50,           // 50 SAYN minimum

faucetEnabled: true,

faucetAmount: 2000,    // 2000 SAYN per request

// Gas configuration

gasLimits: {

minGasPrice: 2,        // Higher minimum

maxGasPerTx: 10000000

}

};
```

Troubleshooting

Port Already in Use

```
Error: listen EADDRINUSE: address already in use :::3000
```

Solution:

```
# Use different port
```

```
PORT=3001 npm run testnet
```

Or kill process using port 3000

```
lsof -ti:3000 | xargs kill
```

CLI Not Found

```
sayman: command not found
```

Solution:

```
# Reinstall CLI
```

```
cd cli
```

```
npm link
```

Or use npx

```
npx sayman wallet create
```

Database Corruption

```
Error: LEVEL_CORRUPTION
```

Solution:

```
# Delete and rebuild  
  
rm -rf data/  
  
npm start
```

Out of Gas Errors

```
Error: Out of gas
```

Solution:

```
# Increase gas limit  
  
sayman send 0x... 100 --gas-limit 100000
```

Or estimate first

```
sayman estimate TRANSFER '{"from":"...", "to":"...", "amount":100}'
```

Invalid Nonce

```
Error: Invalid nonce. Expected: 5, Got: 3
```

Solution:

```
# Check current nonce  
  
sayman balance
```

Transactions must be sequential

If nonce is 5, next transaction must

Rate Limit Exceeded

```
Error: Rate limit exceeded. Please wait.
```

Solution: Wait 1 minute and retry.

Uninstalling

Remove CLI

```
cd cli  
  
npm unlink
```

Remove Node Modules

```
rm -rf node_modules  
  
rm -rf cli/node_modules
```

Remove Database

```
rm -rf data/
```

Remove Wallet

```
rm -rf ~/.sayman/
```

Next Steps

Learn More

1. Read `README.md` for detailed features
2. Check `NETWORK_INFO.md` for network details
3. Review smart contract examples
4. Explore API endpoints

Join the Network

1. Run a validator node
2. Stake tokens
3. Deploy contracts

4. Help secure the network

Contribute

1. Report issues
2. Submit improvements
3. Write documentation
4. Create tutorials

Support

- **Documentation:** README.md
- **Network Info:** `npm run docs`
- **CLI Help:** `sayman --help`
- **Issues:** GitHub Issues

Congratulations! You've successfully installed Sayman Phase 21.

Start exploring with:

```
sayman network
```

```
sayman validators
```

```
sayman balance
```

Happy blockchain building! 🚀🔗

<div style="page-break-after: always;"></div>

SAYMAN Blockchain: Phase-by-Phase Roadmap Comparison

This document provides a simple, high-level comparison of the development roadmap of the **SAYMAN Blockchain** and **PUKY Wallet** ecosystem from Phase 1 to Phase 20.



Summary Comparison Table

| Phase | Milestone Name | Core Deliverables (Simplified) |

| :--- | :--- | :--- |

| **Phase 1 | Genesis & PoS Core** | Created the genesis block, PoS consensus structures, and basic block hashing. |

| **Phase 2 | CLI, Wallet, & SDK** | Built the command-line wallet manager, SDK, and keypair generator. |

| **Phase 3 | P2P Networking** | Launched peer-to-peer gossip protocol for network handshake and block sharing. |

| **Phase 4 | JS Smart Contracts** | Deployed a sandboxed JavaScript execution VM for smart contract runs. |

| **Phase 5 | Gas Model** | Charged execution fees ("sprinkles" gas) for CPU instructions and writes. |

| **Phase 6 | Merkle State Trees** | Implemented Merkle Patricia Trees for state-root verification and proofs. |

| **Phase 7 | Halving & Slashing** | Added block reward halving and validator double-sign slashing. |

| **Phase 8 | Database Persistence** | Integrated LevelDB to save blockchain blocks and state to disk. |

| **Phase 9 | Validator Reputation** | Built validator reputation scoring and event logger. |

| **Phase 10 | Report Systems** | Integrated report creation and community verification transaction types. |

| **Phase 11 | Explorer UI** | Developed a browser-based blockchain explorer with stats and block feeds. |

| **Phase 12 | Fee Sponsorship** | Allowed dApp creators to sponsor user transaction gas fees (free runs). |

| **Phase 13 | Fork Protection** | Fixed P2P sync race conditions and introduced fork-choice rules. |

| **Phase 14 | Multi-Layer & DeFi** | Added L2 Rollups, Sidechain factory, custom Token/NFT factories, and Uniswap AMM DEX. |

| **Phase 15 | Android APK Build** | Configured Capacitor/Gradle to compile the PUKY Wallet APK. |

| **Phase 16 | Sandbox Hardening** | Frozen JS prototypes in VM to prevent escapes; added PDF doc generator. |

| **Phase 17 | Constructor State Fix** | Fixed VM constructor execution (prototype pre-binding); added Dark Theme & SPA routing. |

| **Phase 19 | Pipelined Concurrency & Database Security** | Implemented parallel scheduling, snapshot-accelerated block replay (no more event-loop blocking), atomic block persistence (preventing data loss/corruption), self-connection blacklisting, and WebSocket leak fixes. |

| **Phase 20 | Robust Archive Recovery & Integrated Explorer** | Implemented concurrency-safe archive lock, exponential backoff (starting at 5s up to 5 min), pipelined sync checks to pause archiver during active block download, validation cache fixes, and dedicated transaction page with validator rewards filters. |

| **Phase 21 | EVM JSON-RPC Wallet Integration, Parallel Archive Sync & Memory Safety** | Full EVM JSON-RPC 2.0 server for MetaMask/Trust Wallet/Coinbase Wallet/all wallets, batched parallel archive sync (10 concurrent chunks), ChainProxy 100-block LRU memory model backed by LevelDB (no OOM ever), O(1) indexed block/tx/address DB lookups, root NETWORK_INFO.md wallet guide. |

In-Depth Details per Phase

Phase 1: Genesis & PoS Core

- * **Consensus:** Developed Proof-of-Stake core logic where validator slot allocation depends on stake.
- * **Blocks:** Formulated block validation schemas (hash, validator signature, difficulty).

Phase 2: CLI, Wallet, & SDK

- * **Command Line:** Enabled key generation, sending transactions, and checking balances via terminal.
- * **JS SDK:** Created cryptographic wrappers for signing transactions in browser clients.

Phase 3: P2P Networking

- * **Gossip Protocol:** Connected nodes via WebSockets to broadcast new blocks/transactions.
- * **Syncing:** Built download handshakes so late-joining nodes catch up with the chain.

Phase 4: JS Smart Contracts

- * **V8 Sandbox:** Smart contracts are written in plain JavaScript and executed in isolated sandboxed contexts (no EVM assembly needed).

Phase 5: Gas Model

- * **Instruction Cost:** Prevents infinite loops by charging gas per CPU tick and storage write.
- * **Denomination:** Stored on-chain values in base units ("sprinkles" where 1 SAYN = 10^8 sprinkles).

Phase 6: Merkle State Trees

- * **Verification:** Computes cryptographic state roots so light clients can verify balances and proofs without reading the entire chain.

Phase 7: Halving & Slashing

- * **Inflation Control:** Automatically splits block rewards in half every ~2 years.
- * **Slashing:** Burned stakes of rogue validators double-signing blocks.

Phase 8: Database Persistence

- * **Disk Storage:** Switched from in-memory arrays to LevelDB database storage so nodes survive restarts.

Phase 9: Validator Reputation

- * **Reputation Points:** Validators earn reputation points for valid blocks, displayed in explorer.
- * **Event Log:** Emitted on-chain execution events for dApp listeners.

Phase 10: Report Systems

- * **Use Case Integration:** Added specialized transaction types to create and verify decentralized reports.

Phase 11: Explorer UI

- * **Visuals:** Launched dashboard charts showing blocks, gas limits, validators, and transactions.

Phase 12: Fee Sponsorship

- * **Gas Abstraction:** Enabled dApps to pay transaction fees for users, removing the requirement for users to hold native SAYN tokens.

Phase 13: Fork Protection

- * **Sync Resolution:** Re-syncs only when a valid longer chain is presented, resolving network splits.

Phase 14: Multi-Layer & DeFi

- * **L2 Rollups:** Pushed commitments from sidechains/L2s back to the L1 base chain.
- * **DeFi Primitives:** Deployed standard AMM swap pools, ERC20 tokens, and NFT collections.

Phase 15: Android APK Build

- * **APK package:** Recompiled Capacitor resources to bundle the PUKY Wallet into a mobile APK binary.

Phase 16: Sandbox Hardening

- * **VM Security:** Frozen global prototype objects to block V8 context breakouts.

Phase 17: Constructor State Fix

- * **VM Reliability:** Bound state helpers to contract prototypes **before** constructor instantiation to fix constructor runtime crashes and enable contract state updates.
- * **UX/UI:** SPA routing fallbacks for `/block/:id` , `/tx/:hash` , `/contract/:address` and built-in Moon/Sun dark theme.

Phase 19: Pipelined Concurrency & Database Security

- * **Parallel Execution:** Dependency analysis on read/write sets schedules independent transactions into layers to run concurrently.
- * **Atomic Block Persistence:** Saves blocks individually (`block:${index}`) to prevent database corruption or full history loss during power cuts or peer crashes. Migrates legacy chain keys on startup.
- * **Snapshot Rollbacks:** Accelerates fork syncing by loading historical state snapshots instead of replaying the entire chain from block 0.
- * **Non-Blocking Replays:** Yields execution control periodically to the event loop during replay, preventing heartbeats from timing out.
- * **Self-Connection Blacklist:** Keeps nodes from trying to connect to themselves, resolving reconnect loop spam.
- * **Socket Safety:** Patched P2P WebSocket cleanups to close connections on removal, preventing file descriptor leaks.

Phase 20: Robust Archive Recovery & Integrated Explorer

- * **Archive Lock Mutex:** Added a processing flag to prevent concurrent archive runs when blocks are queued rapidly, solving resource exhaustion.
- * **Exponential Backoff:** Introduced automatic retry backoff starting at 5 seconds and scaling up to 5 minutes for remote Github archive errors, replacing infinite failure loops.
- * **Pipelined Syncing Pause:** Pauses the archive system during active synchronization from peers to prioritize CPU, network bandwidth, and memory for blockchain catchup.

- * **Cache Safety Fix:** Prevented permanent verification failures caused by caching false results on database/system read errors.
- * **Dedicated Transaction Explorer:** Expanded explorer with a transactions view, rewards filter, and real-time statistics.
- * **Repository Cleanups:** Removed deprecated client, SDK, faucet, and faucet-site folders to isolate base chain components.

Phase 21: EVM JSON-RPC Wallet Integration, Parallel Archive Synchronization & Memory Safety

- * **EVM JSON-RPC Server:** Full JSON-RPC server (supporting `eth_chainId`, `net_version`, `eth_blockNumber`, `eth_getBlockByNumber`, `eth_getBlockByHash`, `eth_getBalance`, `eth_getTransactionCount`, `eth_gasPrice`, `eth_estimateGas`, `eth_sendRawTransaction`, `eth_getTransactionByHash`, `eth_getTransactionReceipt`, etc.) allowing MetaMask, Trust Wallet, Coinbase Wallet, and **all other EVM-compatible wallets globally** to connect, display balances, and send/receive transactions natively using standard Ethereum RPC protocol.
- * **Parallel Archive Synchronization:** Replaced old 15-concurrency-all-at-once pool with sequential batched parallel download (10 chunks per batch), restoring nodes to target height in seconds with controlled memory footprint. Each batch downloads, writes to LevelDB, and updates the chain proxy length atomically before moving to the next batch — preventing any OOM burst.
- * **ChainProxy Memory Model:** Replaced the in-memory `this.chain = []` array with a `createChainProxy(blockchain)` Proxy that wraps a 100-block LRU cache backed by LevelDB. Blocks older than 100 are automatically evicted from memory. This eliminates the Render 512 MB OOM crash permanently — node can handle **billions of blocks** without accumulating RAM.
- * **LevelDB Index Tables:** Every `chain.push(block)` now atomically writes: `block:<N>` (full block JSON), `hash:<hash>` (block index lookup), `tx:<txId>` (block+tx index lookup), and `addr:<address>:<blockIndex>:<txIndex>` (per-address transaction index). All API queries (`/blocks`, `/transactions/:id`, `/address/:address`, `/search/:query`) now use these O(1) indexed lookups instead of O(N) full chain scans.
- * **Async Route Handlers:** All Express route handlers in `api/routes.js` converted to `async/await` using `getBlock(index)` which serves from LRU cache or fetches from LevelDB on demand, ensuring zero memory pressure from serving large block ranges.
- * **p2p/server.js Optimized:** `_handleGetBlocks`, `_handleNewBlock`, `_handleNewTransaction`, `_handleBlocks` all updated to use `async getBlock()` and the LevelDB `tx:<id>` index — no more O(N) scans over the full chain when handling peer requests.
- * **NETWORK_INFO.md Root File:** Moved network documentation from `docs/NETWORK_INFO.md` to root `NETWORK_INFO.md` with complete MetaMask wallet connection guide, all JSON-RPC methods, network

parameters, bootstrap peers, API endpoints, and economics data.